

Relatório de Laboratório

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Características de repositórios populares + Setup do Kanban
Grupo (trio)	Caio Victor Kodato Teixeira · Henrique Volponi · Jonas Martins
Link do repositório / GitHub Projects	github.com/CaioVKodato/lab-experimentacao-software · github.com/users/CaioVKodato/projects/6
Data de entrega	27/08/2026

1. Introdução

Este laboratório investiga características de repositórios open-source populares no GitHub. Coletamos os 1.000 repositórios com mais estrelas e respondemos sete questões de pesquisa sobre maturidade, contribuição externa, releases, atualização de código, linguagem primária, taxa de issues fechadas e o cruzamento dessas métricas por linguagem.

Hipóteses informais (antes da análise):

RQ01 — repositórios populares tendem a ser antigos; RQ02 — recebem muita contribuição externa via PRs; RQ03 — lançam releases com frequência, mas não em todos; RQ04 — são atualizados com frequência; RQ05 — concentram-se nas linguagens mais usadas no GitHub; RQ06 — quem usa o tracker de issues tem alta taxa de fechamento; RQ07 — linguagens do top-10 Octoverse têm mais PRs, releases e pushes recentes.

Inovação do grupo (30%): RQ extra — popularidade (stars) se correlaciona com o percentual de issues fechadas? Hipótese: não; o detalhamento está na seção 3.6.

2. Contexto

O Lab01 é o primeiro laboratório do semestre e inaugura o Kanban do grupo (GitHub Projects v2), mantido até o Lab05. Na S01 montamos o cliente GraphQL e coletamos 100 repositórios; na S02 escalamos para 1.000 e validamos os dados; na S03 analisamos e visualizamos as sete RQs.

Objeto de estudo: os 1.000 repositórios públicos com mais estrelas no GitHub. Para linguagens mais populares (RQ05 e RQ07) adotamos o ranking do GitHub Octoverse 2025, mantido de forma consistente em todo o laboratório.

3. Metodologia

3.1 Principais Desafios

Rate limit da API GraphQL ao coletar mil repositórios — mitigado com leitura de `rateLimit`, `retry` e pausa quando o `remaining` fica baixo. Paginação da Search API (teto de 1.000 hits por query e nós nulos) exigiu janelas sucessivas por faixa de estrelas. O GitHub Projects v2 não expõe histórico de mudança de coluna via API, o que obrigou snapshots manuais ao fim de cada sprint. Ambiguidade em RQ04 entre `pushedAt` e `updatedAt` foi resolvida adotando `pushedAt` como métrica de atualização de código.

3.2 Tomadas de Decisão

WIP em Doing = 6 cartões (~2 por integrante no trio), para limitar paralelismo e forçar Review. RQ04 usa `pushedAt`, não `updatedAt`. RQ06 conta issues via GraphQL (sem PRs); repositórios com `total_issues = 0` ficam fora da análise de `closed_ratio`. Outliers identificados pelo critério de Tukey ($1,5 \times \text{IQR}$). Coleta feita com script próprio em Python, sem bibliotecas que encapsulam a API do GitHub.

3.3 Etapas

S01 — Caio: infraestrutura GraphQL, coleta de 100 repos, snapshot do board. Henrique: validação das fatias de RQs. Jonas: hipóteses informais.

S02 — Caio: paginação e coleta dos 1.000 repos, primeira versão do relatório, snapshot. Henrique: validação de consistência dos dados. Jonas: consolidação das hipóteses.

S03 — Caio: análise RQ01–RQ03. Jonas: análise RQ04–RQ06. Henrique: cruzamento RQ07.

Configuração do processo: colunas Backlog → To Do → Doing → Review → Done; WIP = 6; snapshots em snapshots/ (S01: 2026-08-13, S02: 2026-08-21, S03: 2026-08-27). Board: github.com/users/CaioVKodato/projects/6

3.4 Ferramentas

Python 3.12; requests e python-dotenv (cliente GraphQL próprio em `src/github/`); matplotlib e scipy (análise e gráficos); pandas (RQ04–RQ06). Processo: GitHub Projects v2 — github.com/users/CaioVKodato/projects/6. Repositório: github.com/CaioVKodato/lab-experimentacao-software

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data da coleta – createdAt	Dias	GraphQL (createdAt)
RQ02	PRs aceitas	<code>pullRequests(states: MERGED).totalCount</code>	Contagem	GraphQL
RQ03	Releases	<code>releases.totalCount</code>	Contagem	GraphQL
RQ04	Última atualização	Data da coleta – pushedAt	Dias	GraphQL (pushedAt)
RQ05	Linguagem primária	<code>primaryLanguage.name</code>	Categoria	GraphQL
RQ06	Issues fechadas / total	<code>closed / (closed + open)</code> ; exclui total = 0	Proporção	GraphQL issues
RQ07	Cruzamento por linguagem	Medianas RQ02–04 por linguagem vs Octoverse	—	Script Python
Extra	Stars × closed_ratio	Spearman entre stars e closed_ratio	ρ	Script Python (S01)

3.6 Inovações Propostas pelo Grupo (30% da nota)

RQ extra (S01): stars correlacionam com closed_ratio? Correlação de Spearman entre stars e closed_ratio nos 100 repos iniciais (89 válidos após excluir total_issues = 0). Resultado: Spearman $\rho \approx -0,05$ — correlação nula. A hipótese se confirma: popularidade não prediz

saúde do tracker. Documentação: docs/rq_extra_stars_vs_closed_ratio.md. Também adotamos paginação por janela de estrelas e snapshots GraphQL do Kanban como série temporal para os Labs 04 e 05.

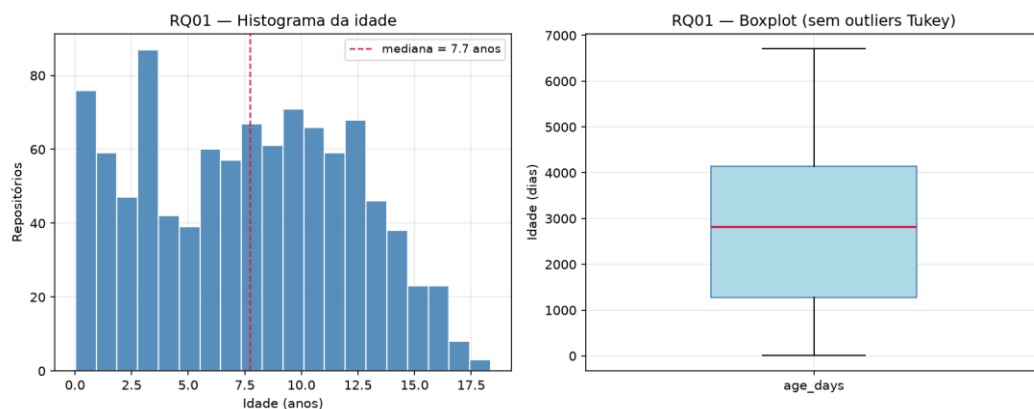
4. Resultados

4.1 Coleta de Dados

Amostra final: 1.000 repositórios válidos em data/repositories_top1000.csv. Campos das RQs 01–05 e 07 completos (N = 1.000). RQ06: 957 repositórios com pelo menos uma issue; 43 excluídos (total_issues = 0). Outliers: 124 em merged_prs, 93 em releases, 196 em days_since_push (Tukey). Três snapshots do Kanban disponíveis (ago/2026).

4.2 Visualização Gráfica

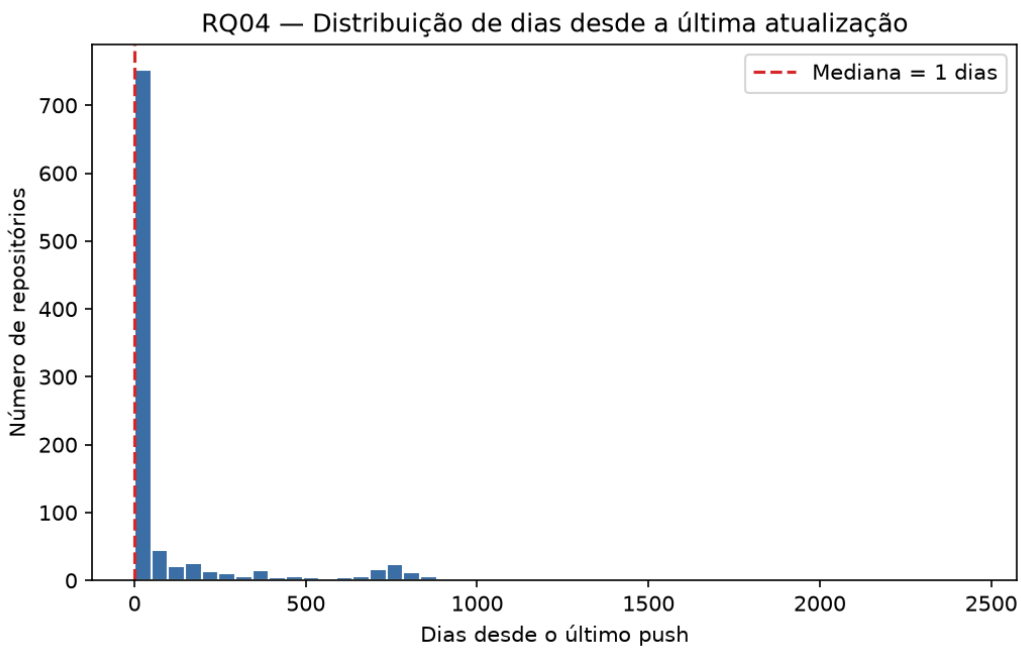
RQ01 — Sistemas populares são maduros/antigos? Mediana: 2.826 dias (~7,7 anos).



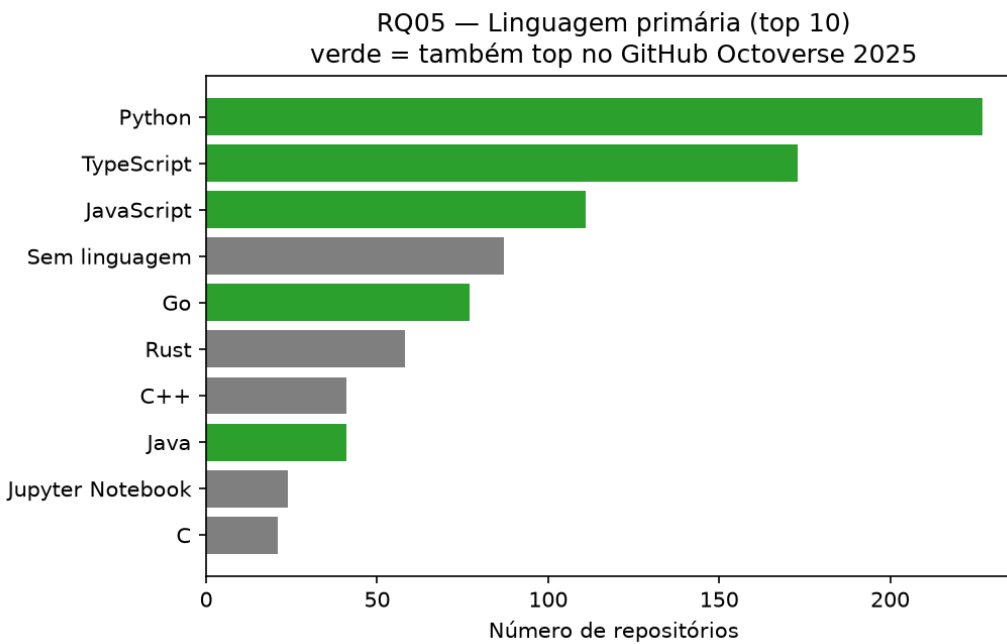
RQ02 — Sistemas populares recebem muita contribuição externa? Mediana: 768 PRs aceitas.

RQ03 — Sistemas populares lançam releases com frequência? Mediana: 40; 27,7% sem release.

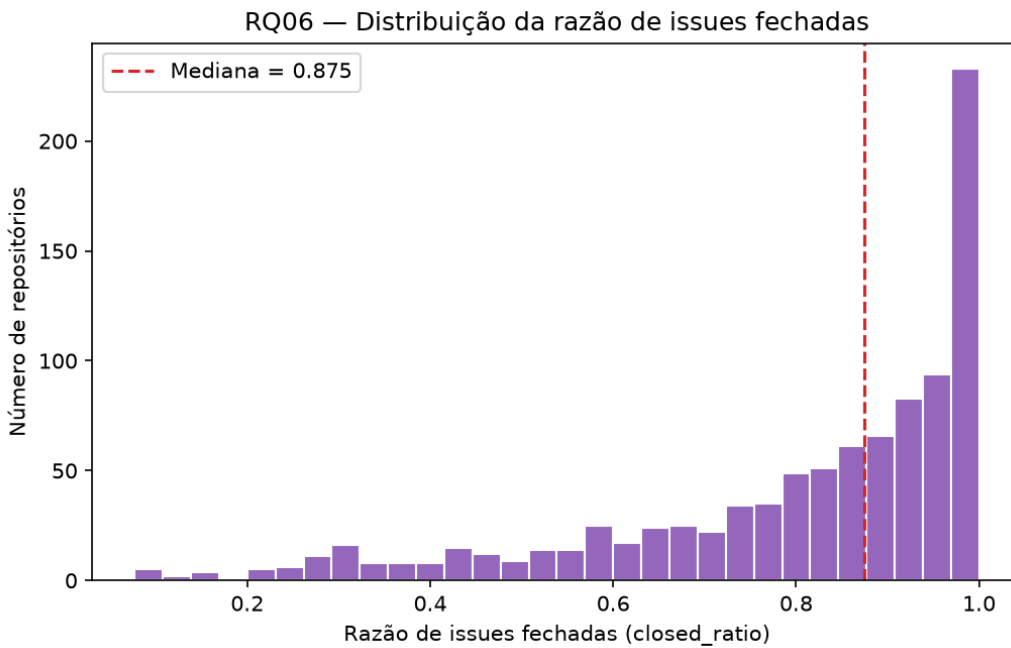
RQ04 — Sistemas populares são atualizados com frequência? Mediana: 1 dia desde o push.



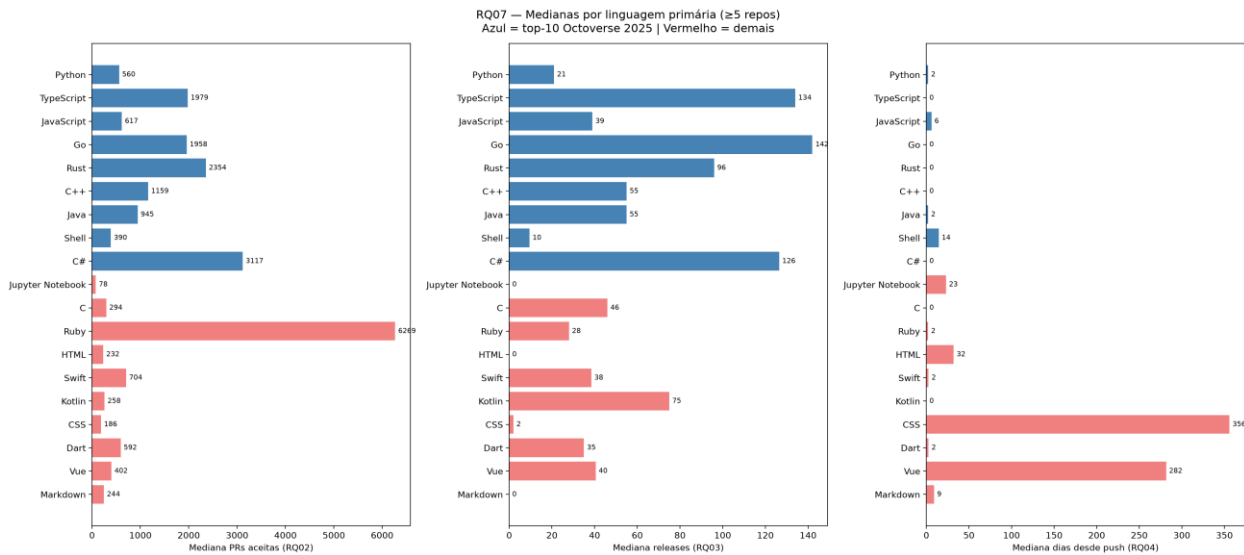
RQ05 — Sistemas populares usam linguagens populares? Python 22,7%, TypeScript 17,3%, JS 11,1%.



RQ06 — Alto percentual de issues fechadas? Mediana: 87,5% (N = 957).



RQ07 — Linguagens populares têm mais PRs, releases e atualizações?



4.3 Discussão

RQ01 — confirmada no centro da distribuição (mediana ~7,7 anos), com ressalva de 8,2% com menos de 1 ano.

RQ02 — confirmada para o típico (mediana 768 PRs); 2% sem PR no GitHub.

RQ03 — parcial: 27,7% sem nenhuma release.

RQ04 — confirmada: mediana de 1 dia desde o último push.

RQ05 — confirmada: domínio de Python, TypeScript e JavaScript.

RQ06 — confirmada: mediana de 87,5% de issues fechadas.

RQ07 — confirmada nas três métricas cruzadas por linguagem.

Amostra mistura frameworks ativos com listas e tutoriais que viralizam sem ciclo de software

tradicional — isso pesa nas RQs 02, 03 e 07. Ordenar por estrelas favorece ecossistemas grandes. A RQ extra reforça que stars não indicam manutenção de issues.

A inovação não contradiz o enunciado: mostra que popularidade e `closed_ratio` são dimensões independentes, enquanto o restante descreve maturidade, atividade e engajamento.

5. Conclusão

Repositórios muito estrelados costumam ser antigos, ativos e com comunidade grande, mas a amostra não é homogênea: listas e guias distorcem releases e PRs. As hipóteses do enunciado se confirmam no agregado, com RQ03 apenas parcial. Limitações: viés da Search API, mistura de tipos de projeto e métricas que dependem do uso do GitHub (PRs, Releases). Com mais tempo, segmentaríamos por tipo de repositório. A RQ extra e a série de snapshots valem expandir nos laboratórios seguintes.

5. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- GITHUB. Octoverse 2025. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>
- GITHUB. GraphQL API documentation. Disponível em: <https://docs.github.com/en/graphql>
-